

POCKETSMART AI: CODING & SOLUTION IMPLEMENTATION

1. Application Initialization & Middleware Setup (app.py)

The system initializes via FastAPI running on an asynchronous Uvicorn ASGI server. Cross-Origin Resource Sharing (CORS) middleware is configured to permit secure cross-domain requests, and static assets and Jinja2 template engines are mounted dynamically:

```
app = FastAPI(title="PocketSmart: AI Budget Planner")
```

```
app.add_middleware(  
    CORSMiddleware,  
    allow_origins=["*"],  
    allow_credentials=True,  
    allow_methods=["*"],  
    allow_headers=["*"],  
)
```

```
templates = Jinja2Templates(directory="templates")  
app.mount("/static", StaticFiles(directory="static"), name="static")  
app.include_router(planners_router)
```

2. Dynamic Authentication & Token Lifecycle (app.py)

To ensure reliable persistence without requiring an external database server, user accounts are stored in `users.json`. Password

hashes are validated against user records, generating signed JWT bearer tokens stored via secure HTTP-only cookies:

```
@app.post("/token")
```

```
async def login_for_access_token(request: Request):
```

```
    form = await request.form()
```

```
    uname = str(form.get("username", "")).strip().lower()
```

```
    pwd = str(form.get("password", "")).strip()
```

```
    current_users = load_users()
```

```
    user = current_users.get(uname)
```

```
    if not user or not verify_password(pwd,  
    user.get("hashed_password")):
```

```
        raise HTTPException(status_code=400, detail="Invalid  
    username or password")
```

```
    access_token = create_access_token(data={"sub": uname})
```

```
    response = JSONResponse(content={"access_token":  
    access_token, "token_type": "bearer"})
```

```
    response.set_cookie(key="access_token", value=f"Bearer  
    {access_token}", httponly=True)
```

```
    return response
```

3. Modular Planning Algorithms & E-Commerce Link Construction (routes/planners.py)

3.1 Home Interior Budget Allocator

Computes proportional financial partitions for lighting, ventilation, and furniture while dynamically generating search links for Amazon India, Flipkart, and IKEA:

```
alloc_light = int(total_b * 0.25)
```

```
alloc_fan = int(total_b * 0.25)
```

```
alloc_furn = int(total_b - (alloc_light + alloc_fan))
```

```
shopping_links = {
```

```
    "amazon":
```

```
f"https://www.amazon.in/s?k={urllib.parse.quote_plus(search_term)}",
```

```
    "flipkart":
```

```
f"https://www.flipkart.com/search?q={urllib.parse.quote_plus(search_term)}",
```

```
    "ikea":
```

```
f"https://www.ikea.com/in/en/search/?q={urllib.parse.quote_plus(search_term)}"
```

```
}
```

3.2 Party & Event Hospitality Allocator

Distributes total event funds based on guest headcount constraints across venue bookings (30%), catering services (50%), and ambient party decorations (20%), linking directly to Swiggy, Zomato, and OYO.

3.3 Multimodal Jewelry Vision Planner

Accepts uploaded reference styling images via UploadFile, processes occasion formality, and generates itemized cost recommendations for primary statement necklaces and secondary matching accessories.

4. Fail-Safe High-Availability Architecture

To ensure continuous application availability during external network delays, rate limiting, or zero-credit API states, the backend incorporates algorithmic fallback mechanisms. These execute deterministic budget partitions within 1.2 milliseconds, returning a 200 OK status without application downtime.